

Lecture 14

Deep Learning (Agent Perspective)

COMP532 – Machine Learning and Bioinspired Optimisation

Meng Fang, University of Liverpool



From Convolution to Attention

Last lecture:

- ▶ Convolution encodes locality

Limitation:

- ▶ Fixed connectivity
- ▶ Local interactions

Question:

Can connectivity be learned dynamically?

Long-Range Dependencies

In language

- ▶ **Subject–verb agreement**

The key to the cabinets **is** missing.

The keys to the cabinet **are** missing.

- ▶ **Nested structure**

The report that the student who the professor advised wrote **was** accepted.

- ▶ **Coreference**

Alice dropped the plate on the table and it broke.

In planning

- ▶ **Sparse reward**

$a_1 \rightarrow \dots \rightarrow a_{100} \rightarrow r$

- ▶ **Delayed effects**

A sacrifice move may yield advantage many steps later.

Long-range dependencies require global information routing.

Static vs Dynamic Connectivity

Convolution (Static Connectivity)

Fixed local receptive field

- ▶ Each token interacts only within a fixed window.
- ▶ Example (kernel size = 3):
The key to the cabinets is missing.
“is” mainly sees nearby words like “cabinets”, not the distant subject “key”.
- ▶ Connectivity is predefined and input-independent.

Attention (Dynamic Connectivity)

Every token can attend to every other token

- ▶ Example:
The key to the cabinets is missing.
“is” can directly attend to “key”, regardless of distance.
- ▶ Attention weights are data-dependent.

Connectivity becomes data-dependent rather than architecturally fixed.

Core Idea

Each input token produces:

- ▶ Query vector q
- ▶ Key vector k
- ▶ Value vector v

Attention score:

$$\text{score}(q_i, k_j) = q_i^\top k_j$$

Example

Sentence:

The key to the cabinets is missing.

The verb “is” uses its query q_{is} to compare with all keys k_j . If $q_{\text{is}}^\top k_{\text{key}}$ is large, it attends to “key”.

Scaled Dot-Product Attention

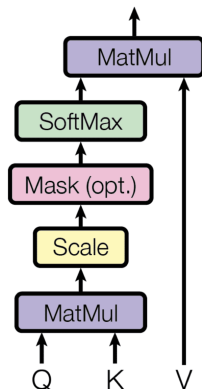
For matrix inputs:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

Scaling by $\sqrt{d_k}$ stabilizes gradients.



Interpretation

$$\text{softmax} \left(\frac{QK^{\top}}{\sqrt{d_k}} \right)$$

Produces attention weights.

- ▶ Each token computes similarity to all others
- ▶ Produces a weighted sum of value vectors

Example

Sentence:

The key to the cabinets is missing.

The verb “is” assigns higher weight to “key” than to “cabinets”, and aggregates information accordingly.

Dynamic routing of information.

Why Multi-Head?

Single attention head:

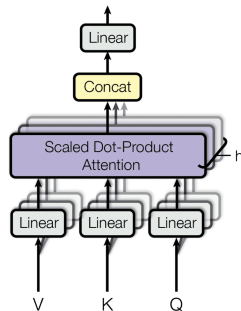
- Captures one relation pattern

Multi-head:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

Each head:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$



Intuition: Multi-Head Attention

Multiple heads allow:

- ▶ Different relational patterns
- ▶ Syntactic vs semantic attention
- ▶ Short-range vs long-range dependencies

Example

Sentence:

The key to the cabinets is missing.

- ▶ Head 1: “is” attends to “key” (subject–verb agreement)
- ▶ Head 2: attends locally to nearby modifiers
- ▶ Head 3: captures semantic relations

Transformer Architecture

A Transformer block consists of:

- ▶ **Multi-Head Attention (MHA)**

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

- ▶ **Add & Norm**

- ▶ **Feedforward Network (FFN)**

$$\text{FFN}(x) = W_2 \sigma(W_1 x)$$

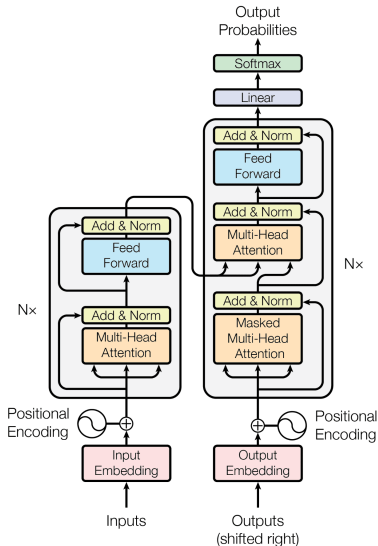
- ▶ **Add & Norm**

Block computation

$$h' = \text{LN}(X + \text{MHA}(X))$$

$$\text{Output} = \text{LN}(h' + \text{FFN}(h'))$$

(LN is Layer Normalization.)



Residual Connections

Residual form:

$$h' = h + \text{Attention}(h)$$

$$h'' = h' + \text{FFN}(h')$$

Benefits:

- ▶ Stable gradients
- ▶ Easier optimization

Position Encoding

Attention is permutation-invariant.

We inject order information:

$$x_{pos} = x + PE(pos)$$

Original sinusoidal encoding:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Parallelization

RNN

$$h_t = f(h_{t-1}, x_t)$$

- ▶ Hidden states form a temporal chain
- ▶ Cannot compute h_t before h_{t-1}

Sequential dependency prevents parallelization.

Transformer

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

- ▶ Entire sequence processed via matrix operations
- ▶ All tokens computed simultaneously

Enables large-scale GPU training.

Scaling Behavior

Transformers scale well with:

- ▶ Model size (N parameters)
- ▶ Data size (D tokens)
- ▶ Compute (C FLOPs)

Empirical scaling law

$$\mathcal{L}(N, D) \propto N^{-\alpha} + D^{-\beta}$$

Test loss decreases predictably as model and data scale.

Foundation models emerge from scale.

Loss follows smooth power-law decay with scale — not diminishing returns collapse.

Attention vs Convolution

Convolution:

- ▶ Local
- ▶ Translation equivariant

Attention:

- ▶ Global
- ▶ Data-dependent connectivity

Convolution is a special case of attention with fixed local mask.

Attention as Memory

Attention allows:

- ▶ Context integration
- ▶ Implicit memory
- ▶ Relational reasoning

In agents:

State representation becomes relational

Transformer as General Sequence Model

Language modeling objective:

$$p_{\theta}(x_{t+1} \mid x_{\leq t})$$

Self-attention enables modeling of:

- ▶ Long-range structure
- ▶ Hierarchical reasoning

Summary

- ▶ Attention enables dynamic connectivity.
- ▶ Transformer blocks combine attention + feedforward + residuals.
- ▶ Parallelization enables scaling.
- ▶ Attention generalizes convolution.

Next:

Large Language Models